

Google Cloud

Dashboard

Explore

Paths

Google Cloud Skills Boost for Partners

← Main menu

Build Custom Processors with Document AI

Course · 6 hours · < 1% complete

Course overview

Build Custom Processors with Document AI

Optical Character Recognition (OCR) with Document AI (Python)

Form Parsing with Document AI (Python)

Using Specialized Processors with Document AI (Python)

Uptraining with Document AI Workbench

Custom Document Extraction with Document AI Workbench

Course > Build Custom Processors with Document AI >

Quick tip: Review the prerequisites before you run the lab

End Lab

00:14:49

Caution: When you are in the console, do not deviate from the lab instructions. Doing so may cause your account to be blocked. [Learn more.](#)

Open Google Cloud console

Username

student-b2-e15c0a76852f

Password

qqCVWe1JXnGq

Project ID

qwiklabs-gcp-b2-beba6c1

Lab

1 hour

No cost

Intermediate

★★★★★

ⓘ

This lab may incorporate AI tools to support your learning.

GSP1138

Google Cloud Self-Paced Labs

Lab instructions and tasks

80/100

GSP1138

Overview

Objectives

Setup and requirements

Task 1. Enable the Document AI API

Task 2. Create and test a processor

Task 3. Authenticate API requests

Task 4. Install the client library

Task 5. Make an online processing request

Task 6. Make a batch processing request

Task 7. Make a batch processing request for a

Next >

Overview

[Document AI](#) is a document understanding solution that takes unstructured data (e.g. documents, emails, invoices, forms, etc.) and makes the data easier to understand, analyze, and consume. The API provides structure through content classification, entity extraction, advanced searching, and more.

In this lab, you will perform Optical Character Recognition (OCR) of PDF documents using Document AI and Python. You will explore how to make both Online (Synchronous) and Batch (Asynchronous) process requests.

We will utilize a PDF file of the classic novel "Winnie the Pooh" by A.A. Milne, which has recently become part of the [Public Domain](#) in the United States. This file was scanned and digitized by [Google Books](#).

Objectives

In this lab, you will learn how to perform the following tasks:

- Enable the Document AI API
- Authenticate API requests
- Install the client library for Python
- Use the online and batch processing APIs
- Parse text from a PDF file

Setup and requirements

Before you click the Start Lab button

Read these instructions. Labs are timed and you cannot pause them. The timer, which

This hands-on lab lets you do the lab activities in a real cloud environment, not in a simulation or demo environment. It does so by giving you new, temporary credentials you use to sign in and access Google Cloud for the duration of the lab.

To complete this lab, you need:

- Access to a standard internet browser (Chrome browser recommended).

Note: Use an Incognito (recommended) or private browser window to run this lab. This prevents conflicts between your personal account and the student account, which may cause extra charges incurred to your personal account.

- Time to complete the lab—remember, once you start, you cannot pause a lab.

Note: Use only the student account for this lab. If you use a different Google Cloud account, you may incur charges to that account.

How to start your lab and sign

1. Click the **Start Lab** button. If you need to pay for the lab, a dialog opens for you to select your payment method. On the left is the Lab Details pane with the following:

- The Open Google Cloud console button
- Time remaining
- The temporary credentials that you must use for this lab
- Other information, if needed, to step through this lab

2. Click **Open Google Cloud console** (or right-click and select **Open Link in Incognito Window** if you are running the Chrome browser).

The lab spins up resources, and then opens another tab that shows the Sign in page.

Tip: Arrange the tabs in separate windows, side-by-side.

Note: If you see the **Choose an account** dialog, click **Use Another Account**.

student-02-e15c0a76852f@qwiklabs.net

You can also find the Username in the Lab Details pane.

4. Click **Next**.

5. Copy the **Password** below and paste it into the **Welcome** dialog.

qqCVNe1JXnGq

You can also find the Password in the Lab Details pane.

6. Click **Next**.

Important: You must use the credentials the lab provides you. Do not use your Google Cloud account credentials.

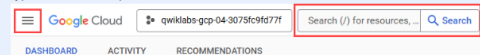
Note: Using your own Google Cloud account for this lab may incur extra charges.

7. Click through the subsequent pages:

- Accept the terms and conditions.
- Do not add recovery options or two-factor authentication (because this is a temporary account).
- Do not sign up for free trials.

After a few moments, the Google Cloud console opens in this tab.

Note: To access Google Cloud products and services, click the **Navigation menu** or type the service or product name in the **Search** field.



Activate Cloud Shell

command-line access to your Google Cloud resources.

1. Click **Activate Cloud Shell** at the top of the Google Cloud console.

2. Click through the following windows:

- Continue through the Cloud Shell information window.
- Authorize Cloud Shell to use your credentials to make Google Cloud API calls.

When you are connected, you are already authenticated, and the project is set to your **Project ID**, `qwiklabs-gcp-02-beba6c153d75`. The output contains a line that declares the **Project ID** for this session:

Your Cloud Platform project in this session is set to qwiklabs-gcp-02-beba

gcloud is the command-line tool for Google Cloud. It comes pre-installed on Cloud Shell and supports tab-completion.

3. (Optional) You can list the active account name with this command:

```
gcloud auth list
```

4. Click **Authorize**.

Output:

```
ACTIVE: *
ACCOUNT: student-02-e15c0a76852f@quwiklabs.net

To set the active account, run:
$ gcloud config set account `ACCOUNT`
```

5. (Optional) You can list the project ID with this command:

```
gcloud config list project
```

Output:

```
[core]
project = qwiklabs-gcp-02-beba5c153d75
```

Note: For full documentation of gcloud, in Google Cloud, refer to [the gcloud CLI overview guide](#).

Task 1. Enable the Document AI API

Before you can begin using Document AI, you must enable the API.

1. Using the Search Bar at the top of the console, search for "Document AI API", then click **Enable** to use the API in your Google Cloud project.



Cloud Document AI API

[Google Enterprise API](#)

Service to parse structured information from unstructured or semi-structured documents using...

2. Use the search bar to search for "Cloud Storage API", if not already enabled, click **Enable**.

3. Alternatively, the APIs can be enabled using the following gcloud commands.

```
gcloud services enable documentai.googleapis.com
gcloud services enable storage.googleapis.com
```

You should see something like this:

```
Operation "operations/..." finished successfully.
```

Now, you can use Document AI!

Click **Check my progress** to verify the objective.

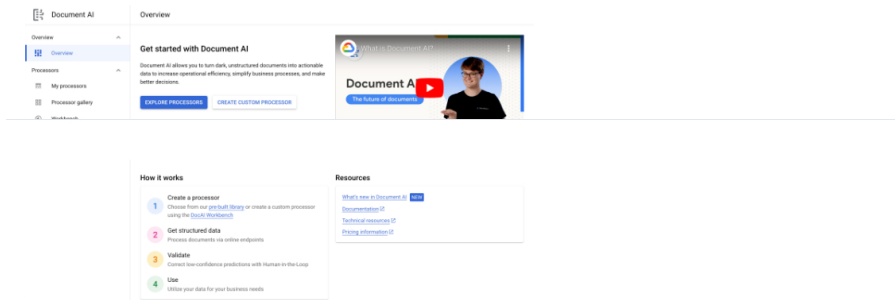
Enable the Document AI API

Task 2. Create and test a processor

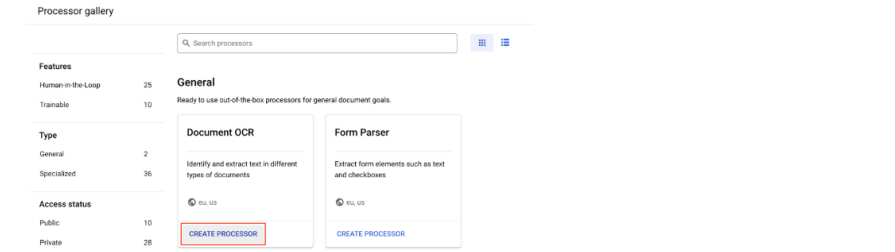
You must first create an instance of the Document OCR processor that will perform the extraction. This can be completed using the Cloud Console or the [Processor Management API](#).

1. From the Navigation Menu, click **View all Products**. Under **Artificial**

Intelligence, select Document AI.

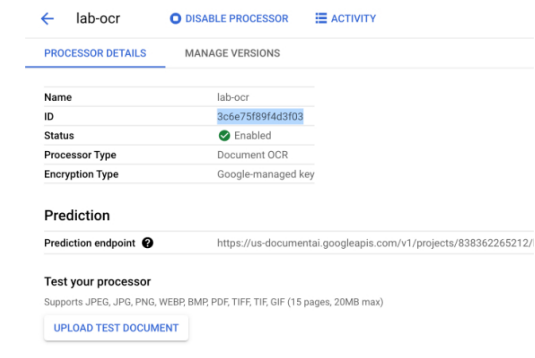


2. Click **Explore Processors**, and inside Document OCR, click **Create Processor**.



4. Click **Create** to create your processor

5. **Copy** your Processor ID. You must use this in your code later.



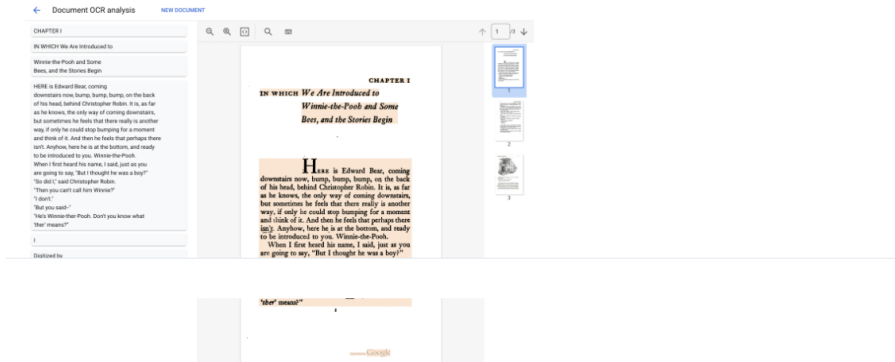
"Winnie the Pooh" by A.A. Milne.

[Download PDF](#)

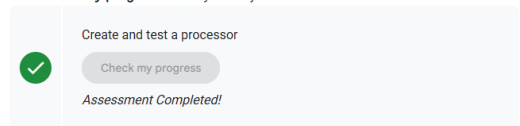
Now you can test out your processor in the console by uploading a document.

7. Click **Upload Test Document** and select the PDF file you downloaded.

Your output should look this:



Click **Check my progress** to verify the objective.



Task 3. Authenticate API requests

In order to make requests to the Document AI API, you must use a *Service Account*. A *Service Account* belongs to your project and it is used by the Python Client library to

and then create credentials you need to authenticate as the service account.

1. First, open a new Cloud Shell window and set an environment variable with your Project ID by running the following command:

```
export GOOGLE_CLOUD_PROJECT=$(gcloud config get-value  
core/project)
```

2. Next, create a new service account to access the Document AI API by using:

```
gcloud iam service-accounts create my-docai-sa \  
--display-name "my-docai-service-account"
```

3. Give your service account permissions to access Document AI, Cloud Storage, and Service Usage by using the following commands:

```
gcloud projects add-iam-policy-binding ${GOOGLE_CLOUD_PROJECT} \  
--member="serviceAccount:my-docai-  
sa@${GOOGLE_CLOUD_PROJECT}.iam.gserviceaccount.com" \  
--role="roles/documentai.admin"
```

```
--member="serviceAccount:my-docai-  
sa@${GOOGLE_CLOUD_PROJECT}.iam.gserviceaccount.com" \  
--role="roles/storage.admin"
```

```
gcloud projects add-iam-policy-binding ${GOOGLE_CLOUD_PROJECT} \  
--member="serviceAccount:my-docai-  
sa@${GOOGLE_CLOUD_PROJECT}.iam.gserviceaccount.com" \  
--role="roles/serviceusage.serviceUsageConsumer"
```

4. Create credentials that your Python code uses to login as your new service account. Create these credentials and save it as a JSON file `~/key.json` by using the following command:

```
gcloud iam service-accounts keys create ~/key.json \  
--iam-account my-docai-  
sa@${GOOGLE_CLOUD_PROJECT}.iam.gserviceaccount.com
```

5. Finally, set the `GOOGLE_APPLICATION_CREDENTIALS` environment variable, which is used by the library to find your credentials. The environment variable should be set to the full path of the credentials JSON file you created, by using:

To read more about this form of authentication, see the [guide](#).

Click **Check my progress** to verify the objective.



Authenticate API requests

Check my progress

Assessment Completed!

Task 4. Install the client library

1. In Cloud Shell, run the following commands to install the Python client libraries for Document AI and Cloud Storage.

```
pip install google-cloud-documentai  
pip install google-cloud-storage
```

You should see something like this:

```
...  
Installing collected packages: google-cloud-documentai  
Successfully installed google-cloud-documentai-1.4.1  
.  
.  
Installing collected packages: google-cloud-storage  
Successfully installed google-cloud-storage-1.43.0
```

Now, you're ready to use the Document AI API!

Note: If you're setting up your own Python development environment, you can follow these [guidelines](#).

Upload the sample PDF to Cloud Shell

1. From the Cloud Shell toolbar, click the three dots and select **Upload**.

earlier.

3. Click **Upload**.

4. **Alternatively**, you can also download the PDF from a public Google Cloud Storage Bucket using `gsutil cp`.

```
gsutil cp gs://cloud-samples-  
data/documentai/codelabs/ocr/Winnie_the_Pooh_3_Pages.pdf .
```

Task 5. Make an online processing request

In this step, you'll process the first 3 pages of the novel using the online processing (synchronous) API. This method is best suited for smaller documents that are stored locally. Check out the [full processor list](#) for the maximum pages and file size for each

1. In Cloud Shell, create a file called `online_processing.py` and paste the following code into it:

```
from google.api_core.client_options import ClientOptions  
from google.cloud import documentai_v1 as documentai  
  
PROJECT_ID = "YOUR_PROJECT_ID"  
LOCATION = "YOUR_PROJECT_LOCATION" # Format is 'us' or 'eu'  
PROCESSOR_ID = "YOUR_PROCESSOR_ID" # Create processor in Cloud  
Console  
  
# The local file in your current working directory  
FILE_PATH = "Winnie_the_Pooh_3_Pages.pdf"  
# Refer to https://cloud.google.com/document-ai/docs/file-types  
# for supported file types  
MIME_TYPE = "application/pdf"  
  
# Instantiates a client  
docai_client = documentai.DocumentProcessorServiceClient(  
    client_options=ClientOptions(api_endpoint=f"{LOCATION}-  
documentai.googleapis.com")  
)  
  
# You must create new processors in the Cloud Console first  
RESOURCE_NAME = docai_client.processor_path(PROJECT_ID,  
    LOCATION, PROCESSOR_ID)  
  
# Read the file into memory  
with open(FILE_PATH, "rb") as image:  
    image_content = image.read()  
  
# Load Binary Data into Document AI RawDocument Object  
raw_document = documentai.RawDocument(content=image_content,  
    mime_type=MIME_TYPE)  
  
# Configure the process request  
request = documentai.ProcessRequest(name=RESOURCE_NAME,  
    raw_document=raw_document)  
  
# Use the Document AI client to process the sample form  
result = docai_client.process_document(request=request)  
  
document_object = result.document  
print("Document processing complete.")  
print(f"Text: {document_object.text}")
```

Note: `FILE_PATH` is the name of the file you uploaded to Cloud Shell in the previous step. If you didn't rename the file, it should be `Winnie_the_Pooh_3_Pages.pdf` which is the default value and doesn't need to be changed.

3. Run the code, which will extract the text and print it to the console.

```
python3 online_processing.py
```

You should see the following output:

```
Document processing complete.  
Text: IN WHICH We Are Introduced to  
CHAPTER 1
```

```
CHAPTER 1
Winnie-the-Pooh and Some
Bees, and the Stories Begin
HERE is Edward Bear, coming
downstairs now, bump, bump, bump, on the back
of his head, behind Christopher Robin. It is, as far
as he knows, the only way of coming downstairs,
```

```
and think of it. And then he feels that perhaps there
isn't. Anyhow, here he is at the bottom, and ready
to be introduced to you. Winnie-the-Pooh.
When I first heard his name, I said, just as you
are going to say, "But I thought he was a boy?"
"So did I," said Christopher Robin.
"Then you can't call him Winnie?"
"I don't."
"But you said--"
...
```

Task 6. Make a batch processing request

Now, suppose that you want to read in the text from the entire novel.

- Online Processing has limits on the number of pages and file size that can be sent and it only allows for one document file per API call.

In this section, you will process the entire "Winnie the Pooh" novel with the Document AI Batch Processing API and output the text into a Google Cloud Storage Bucket.

Batch processing uses [Long Running Operations](#) to manage requests in an asynchronous manner, so we have to make the request and retrieve the output in a different manner than online processing. However, the output will be in the same [Document](#) object format whether using online or batch processing.

This section shows how to provide specific documents for Document AI to process. A later section will show how to process an entire directory of documents.

Upload PDF to Cloud Storage

The `batch_process_documents()` method currently accepts files from Google Cloud Storage. You can reference [documentai_v1.types.BatchProcessRequest](#) for more information on the object structure.

1. Run the following command to create a Google Cloud Storage Bucket to store the PDF file and upload the PDF file to the bucket:

```
gcloud storage cp gs://cloud-samples-
data/documentai/codelabs/ocr/Winnie_the_Pooh.pdf
gs://$GOOGLE_CLOUD_PROJECT/
```

Using the batch_process_documents() method

1. Create a file called `batch_processing.py` and paste in the following code:

```
import re
from typing import List

from google.api_core.client_options import ClientOptions
from google.cloud import documentai_v1 as documentai
from google.cloud import storage

PROJECT_ID = "YOUR_PROJECT_ID"
LOCATION = "YOUR_PROJECT_LOCATION" # Format is 'us' or 'eu'
PROCESSOR_ID = "YOUR_PROCESSOR_ID" # Create processor in Cloud
Console
```

```
data/documentai/codelabs/ocr/Winnie_the_Pooh.pdf"
INPUT_MIME_TYPE = "application/pdf"

# Format 'gs://output_bucket/directory'
GCS_OUTPUT_URI = "YOUR_OUTPUT_BUCKET_URI"

# Instantiates a client
docai_client = documentai.DocumentProcessorServiceClient(
    client_options=ClientOptions(api_endpoint=f"{LOCATION}-
documentai.googleapis.com")
)

# The full resource name of the processor, e.g.:
# projects/project-id/locations/location/processor/processor-id
# You must create new processors in the Cloud Console first
RESOURCE_NAME = docai_client.processor_path(PROJECT_ID,
LOCATION, PROCESSOR_ID)
```

```
# Cloud Storage URI for the Input Document
input_document = documentai.GcsDocument(
    gcs_uri=GCS_INPUT_URI, mime_type=INPUT_MIME_TYPE
)

# Load GCS Input URI into a List of document files
```

```
[input_document])
)

# Cloud Storage URI for Output directory
gcs_output_config =
documentai.DocumentOutputConfig.GcsOutputConfig(
    gcs_uri=GCS_OUTPUT_URI
)

# Load GCS Output URI into OutputConfig object
output_config =
documentai.DocumentOutputConfig(gcs_output_config=gcs_output_config)

# Configure Process Request
request = documentai.BatchProcessRequest(
    name=RESOURCE_NAME,
    input_documents=input_config,
    document_output_config=output_config,
)

# Batch Process returns a Long Running Operation (LRO)
operation = docai_client.batch_process_documents(request)

# Continually polls the operation until it is complete.
```

```
projects/PROJECT_NUMBER/locations/LOCATION/operations/OPERATION_ID
print(f'Waiting for operation {operation.operation.name} to
complete...')
operation.result()

# NOTE: Can also use callbacks for asynchronous processing
#
# def my_callback(future):
#     result = future.result()
#
# operation.add_done_callback(my_callback)

print("Document processing complete.")

# Once the operation is complete,
# get output document information from operation metadata
metadata = documentai.BatchProcessMetadata(operation.metadata)

if metadata.state !=
documentai.BatchProcessMetadata.State.SUCCEEDED:
    raise ValueError(f'Batch Process Failed:
{metadata.state_message}')

documents: List[documentai.Document] = []
```

```
storage_client = storage.Client()

# One process per Input Document
for process in metadata.individual_process_statuses:

    # output_gcs_destination format:
    gs://BUCKET/PREFIX/OPERATION_NUMBER/0
    # The GCS API requires the bucket name and URI prefix
    separately
    output_bucket, output_prefix = re.match(
        r"gs://(.*)/(.*)", process.output_gcs_destination
    ).groups()

    # Get List of Document Objects from the Output Bucket
    output_blobs = storage_client.list_blobs(output_bucket,
        prefix=output_prefix)

    # DocAI may output multiple JSON files per source file
    for blob in output_blobs:
        # Document AI should only output JSON files to GCS
        if ".json" not in blob.name:
            print(f'Skipping non-supported file type
{blob.name}')
            continue
```

```
# Download JSON File and Convert to Document Object
document = documentai.Document.from_json(
    blob.download_as_bytes(), ignore_unknown_fields=True
)

documents.append(document)

# Print Text from all documents
# Truncated at 100 characters for brevity
for document in documents:
    print(document.text[:100])
```

2. Replace YOUR_PROJECT_ID, YOUR_PROJECT_LOCATION, YOUR_PROCESSOR_ID, GCS_INPUT_URI and GCS_OUTPUT_URI with the appropriate values for your environment.

```
PROCESSOR_ID = "YOUR_PROCESSOR_ID" # Create processor in Cloud Console
```

```
# Format 'gs://input_bucket/directory'
GCS_INPUT_PREFIX = "gs://cloud-samples-
data/documentai/code-labs/ocr/multi-document"
```

```
GCS_OUTPUT_URI = "YOUR_OUTPUT_BUCKET_URI"
```

```
# Instantiates a client
docai_client = documentai.DocumentProcessorServiceClient(
    client_options=ClientOptions(api_endpoint=f"{LOCATION}-
documentai.googleapis.com")
)

# The full resource name of the processor, e.g.:
# projects/project-id/locations/location/processor/processor-id
# You must create new processors in the Cloud Console first
RESOURCE_NAME = docai_client.processor_path(PROJECT_ID,
LOCATION, PROCESSOR_ID)
```

```
# Cloud Storage URI for the Input Directory
gcs_prefix =
documentai.GcsPrefix(gcs_uri_prefix=GCS_INPUT_PREFIX)
```

```
# Load GCS Input URI into Batch Input Config
input_config =
documentai.BatchDocumentsInputConfig(gcs_prefix=gcs_prefix)
```

```
# Cloud Storage URI for Output directory
gcs_output_config =
```

```
)

# Load GCS Output URI into OutputConfig object
output_config =
documentai.DocumentOutputConfig(gcs_output_config=gcs_output_config)

# Configure Process Request
request = documentai.BatchProcessRequest(
    name=RESOURCE_NAME,
    input_documents=input_config,
    document_output_config=output_config,
)

# Batch Process returns a Long Running Operation (LRO)
operation = docai_client.batch_process_documents(request)

# Continually polls the operation until it is complete.
# This could take some time for larger files
# Format:
projects/PROJECT_NUMBER/locations/LOCATION/operations/OPERATION_ID
print(f"Waiting for operation {operation.operation.name} to
complete...")
operation.result()
```

```
# Get my_callback() object.
# result = future.result()
#
# operation.add_done_callback(my_callback)

print("Document processing complete.")

# Once the operation is complete,
# get output document information from operation metadata
metadata = documentai.BatchProcessMetadata(operation.metadata)

if metadata.state !=
documentai.BatchProcessMetadata.State.SUCCEEDED:
    raise ValueError(f"Batch Process Failed:
{metadata.state_message}")

documents: List[documentai.Document] = []

# Storage Client to retrieve the output files from GCS
storage_client = storage.Client()

# One process per Input Document
for process in metadata.individual_process_statuses:

    # output gcs destination format:
```

```
separately
    output_bucket, output_prefix = re.match(
        r"gs://(.*)/(.*)", process.output_gcs_destination
    ).groups()

    # Get List of Document Objects from the Output Bucket
    output_blobs = storage_client.list_blobs(output_bucket,
prefix=output_prefix)

    # DocAI may output multiple JSON files per source file
    for blob in output_blobs:
        # Document AI should only output JSON files to GCS
        if ".json" not in blob.name:
            print(f"Skipping non-supported file type
{blob.name}")
            continue

        print(f"Fetching {blob.name}")
```

```
# Download JSON File and Convert to Document Object
document = documentai.Document.from_json(
    blob.download_as_bytes(), ignore_unknown_fields=True
)
```

```
# Print Text from all documents
# Truncated at 100 characters for brevity
for document in documents:
    print(document.text[:100])
```

3. Replace the `PROJECT_ID`, `LOCATION`, `PROCESSOR_ID`, `GCS_INPUT_PREFIX` and `GCS_OUTPUT_URI` with the appropriate values for your environment.

- For `GCS_INPUT_PREFIX`, use the URI of the **directory** you uploaded to your bucket in the previous section, i.e `gs://qwiklabs-gcp-02-beba6c153d75/multi-document`.
- For `GCS_OUTPUT_URI`, use the URI of the bucket you created in the previous section, i.e `gs://qwiklabs-gcp-02-beba6c153d75`.

4. Use the following command to run the code, and you should see the extracted text from all of the document files in the Cloud Storage directory.

```
python3 batch_processing_directory.py
```

Your output should look something like this:

```
Document processing complete.
Fetching 16354972755137859334/0/Winnie_the_Pooh_Page_0-0.json
Fetching 16354972755137859334/1/Winnie_the_Pooh_Page_1-0.json
Fetching 16354972755137859334/2/Winnie_the_Pooh_Page_10-0.json
..
Introduction
(I,
F YOU happen to have read another
book about Christopher Robin, you may remember
th
IN WHICH We Are Introduced to
CHAPTER I
Winnie-the-Pooh and Some
Bees, and the Stories Begin
HERE is
..
```

Great! You've successfully used the Document AI Python Client Library to process a directory of documents using a Document AI Processor, and output the results to Cloud Storage.

Click **Check my progress** to verify the objective.

Check my progress

Please copy the sample directory to your Cloud Storage bucket.

Congratulations!

Congratulations! In this lab, you learned how to use the Document AI Python Client Library to process a directory of documents using a Document AI Processor, and output the results to Cloud Storage. You also learned how to authenticate API requests using a service account key file, install the Document AI Python Client Library, and use the Online (Synchronous) and Batch (Asynchronous) APIs to process requests.

Next steps/Learn more

Check out the following resources to learn more about Document AI and the Python

Document AI Python Client Library.

- [The Future of Documents - YouTube Playlist](#)
- [Document AI Documentation](#)
- [Document AI Python Client Library](#)
- [Document AI Samples Repository](#)

Google Cloud training and certification

...helps you make the most of Google Cloud technologies. [Our classes](#) include technical skills and best practices to help you get up to speed quickly and continue your learning journey. We offer fundamental to advanced level training, with on-demand, live, and virtual options to suit your busy schedule. [Certifications](#) help you validate and prove your skill and expertise in Google Cloud technologies.

Manual Last Updated June 13, 2024

Lab Last Tested June 13, 2024

